

PART A

1. Combinational Logic Design

```
-- Combinational Logic Design
-- (ESD book figure 2.4)
-- by Weijun Zhang, 04/2001
--
-- A simple example of VHDL Structure Modeling
-- we might define two components in two separate files,
-- in main file, we use port map statement to instantiate
-- the mapping relationship between each components
-- and the entire circuit.
```

```
-----

library ieee;                                -- component #1
use ieee.std_logic_1164.all;

entity OR_GATE is
port(   X:      in std_logic;
        Y:      in std_logic;
        F2:     out std_logic
);
end OR_GATE;

architecture behv of OR_GATE is
begin
process(X,Y)
begin
    F2 <= X or Y;                            -- behavior des.
end process;
end behv;
```

```
-----

library ieee;                                -- component #2
use ieee.std_logic_1164.all;

entity AND_GATE is
port(   A:      in std_logic;
        B:      in std_logic;
        F1:     out std_logic
);
end AND_GATE;

architecture behv of AND_GATE is
begin
```

```

process(A,B)
begin
    F1 <= A and B;           -- behavior des.
end process;
end behv;

-----

library ieee;                -- top level circuit
use ieee.std_logic_1164.all;
use work.all;

entity comb_ckt is
port(  input1: in std_logic;
       input2: in std_logic;
       input3: in std_logic;
       output: out std_logic
);
end comb_ckt;

architecture struct of comb_ckt is

    component AND_GATE is      -- as entity of AND_GATE
    port(  A:    in std_logic;
           B:    in std_logic;
           F1:   out std_logic
    );
    end component;

    component OR_GATE is       -- as entity of OR_GATE
    port(  X:    in std_logic;
           Y:    in std_logic;
           F2:   out std_logic
    );
    end component;

    signal wire: std_logic;     -- signal just like wire

begin

    -- use sign ">=" to clarify the pin mapping

    Gate1: AND_GATE port map (A=>input1, B=>input2, F1=>wire);
    Gate2: OR_GATE port map (X=>wire, Y=>input3, F2=>output);

end struct;

```

TEST BENCH VHDL CODE FOR COMBINATIONAL LOGIC DESIGN

```

-- Test Bench for comb_ckt.vhd
-- (ESD figure 2.4)
-- by Weijun Zhang, 04/2001
--
-- Testbench is used to ensure the design is working properly
-- according to the specification.
-- assert statements are used to test the wrong value against
-- our desired one. we should test as many cases as possible,
-- particularly, we should include upper and lower limits
-- of the operations.
-----

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;

entity CKT_TB is                                -- empty entity
end CKT_TB;

-----

architecture TB of CKT_TB is

-- declare the whole circuit(entity of comb_ckt.vhd) as a component

component comb_ckt is
port(   input1: in std_logic;
        input2: in std_logic;
        input3: in std_logic;
        output: out std_logic
);
end component;

-- declare all I/O ports from unit under test as signals.
-- signals are usually declared within architecture

signal T_input1, T_input2, T_input3, T_output: std_logic;

begin

    U_UT: comb_ckt port map (T_input1,T_input2,T_input3,T_output);

    process

        variable err_cnt: integer := 0;

    begin

        -- Test case 1

```

```

T_input1 <= '0';
T_input2 <= '0';
T_input3 <= '0';
wait for 10 ns;
assert (T_output==((T_input1 or T_input2) and T_input3))
report "Failed Case1!" severity error;
if (T_output/=((T_input1 or T_input2) and T_input3)) then
    err_cnt := err_cnt +1;
end if;

-- Test case 2
T_input1 <= '1';
T_input2 <= '1';
T_input3 <= '1';
wait for 10 ns;
assert (T_output==((T_input1 or T_input2) and T_input3))
report "Failed Case1!" severity error;
if (T_output/=((T_input1 or T_input2) and T_input3)) then
    err_cnt := err_cnt +1;
end if;

-- Test case 3
T_input1 <= '1';
T_input2 <= '0';
T_input3 <= '1';
wait for 10 ns;
assert (T_output==((T_input1 or T_input2) and T_input3))
report "Failed Case1!" severity error;
if (T_output/=((T_input1 or T_input2) and T_input3)) then
    err_cnt := err_cnt +1;
end if;

-- Test case 4
T_input1 <= '0';
T_input2 <= '1';
T_input3 <= '0';
wait for 10 ns;
assert (T_output==((T_input1 or T_input2) and T_input3))
report "Failed Case1!" severity error;
if (T_output/=((T_input1 or T_input2) and T_input3)) then
    err_cnt := err_cnt +1;
end if;

-- summary of all the tests to see if any errors

if (err_cnt=0) then
    assert false report "Testbench completed successfully!"
    severity note;
else
    assert true

```

```

report "Something wrong, try again pls!"
severity error;
end if;

```

```

wait; -- stop running

```

```

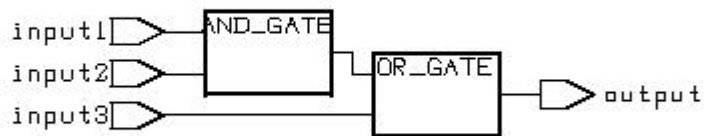
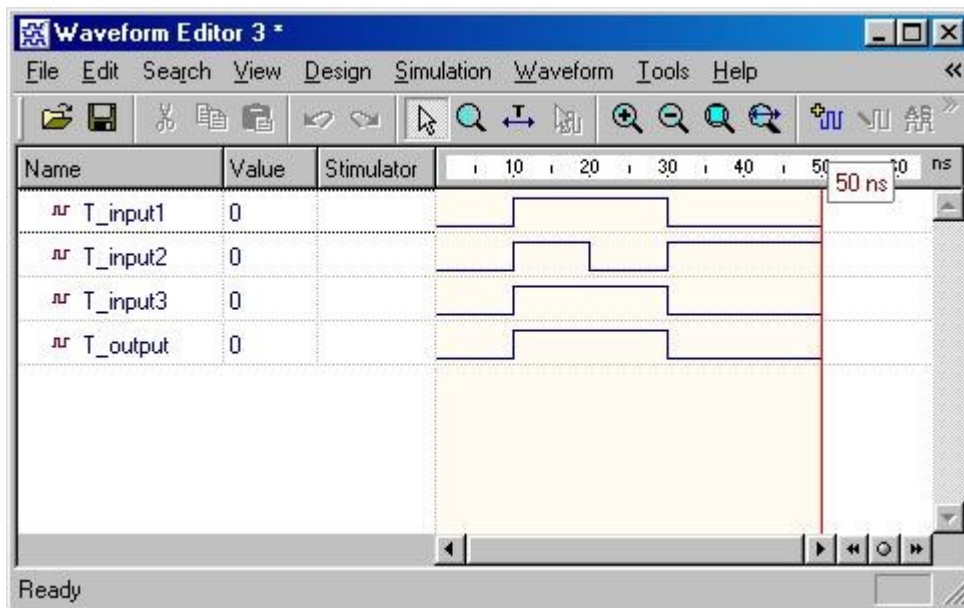
end process;
end TB;

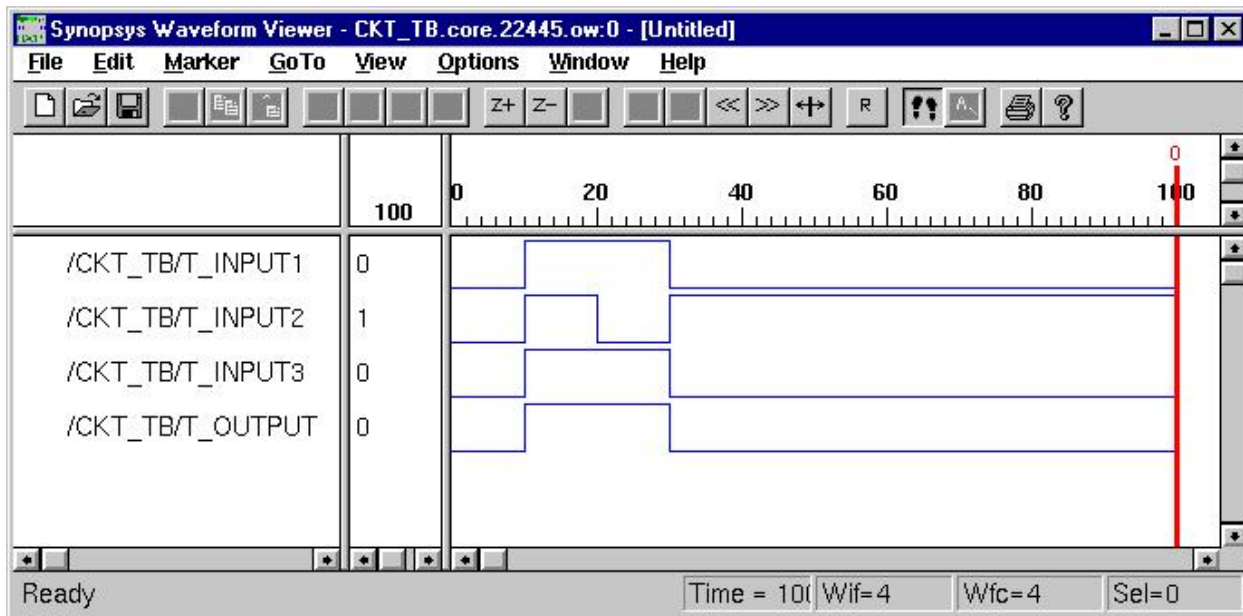
```

```

-----
configuration CFG_TB of CKT_TB is
    for TB
        end for;
end CFG_TB;
-----

```





2. Tri-State Driver

```

-----
-- VHDL model for tri state driver
-- by Weijun Zhang, 05/2001
--
-- this friver often used to control system outputs
-----

```

```

library ieee;
use ieee.std_logic_1164.all;

entity tristate_dr is
port(   d_in:   in std_logic_vector(7 downto 0);
        en:     in std_logic;
        d_out:  out std_logic_vector(7 downto 0)
);
end tristate_dr;

architecture behavior of tristate_dr is
begin

process(d_in, en)
begin
    if en='1' then
        d_out <= d_in;
    else
        -- array can be created simply by using vector
        d_out <= "ZZZZZZZZ";
    end if
end process
end behavior

```

```
        end if;
    end process;

end behavior;
```

TEST BENCH VHDL CODE FOR TRI-STATE DRIVER

```
-----
-- Test Bench for Tri-state Driver
-- by Weijun Zhang, 05/2001
-----
```

```
library ieee;
use ieee.std_logic_1164.all;
```

```
entity TB_tridr is
end TB_tridr;
```

```
architecture TB of TB_tridr is
```

```
    component tristate_dr is
    port( d_in:   in std_logic_vector(7 downto 0);
          en:     in std_logic;
          d_out:  out std_logic_vector(7 downto 0)
    );
    end component;
```

```
    signal T_d_in, T_d_out: std_logic_vector(7 downto 0);
    signal T_en: std_logic;
```

```
begin
```

```
    U_UT: tristate_dr port map (T_d_in, T_en, T_d_out);
```

```
process
begin
```

```
    T_d_in <= "11001100";
    T_en <= '1';
    wait for 20 ns;
    assert(T_d_out = T_d_in) report "Error detected!"
    severity warning;
```

```
    T_en <= '0';
    wait for 20 ns;
    assert(T_d_out = "ZZZZZZZZ") report "Error detected!"
    severity warning;
```

```
    T_en <= '1';
```

```
wait for 10 ns;
```

```
wait;
```

```
end process;
```

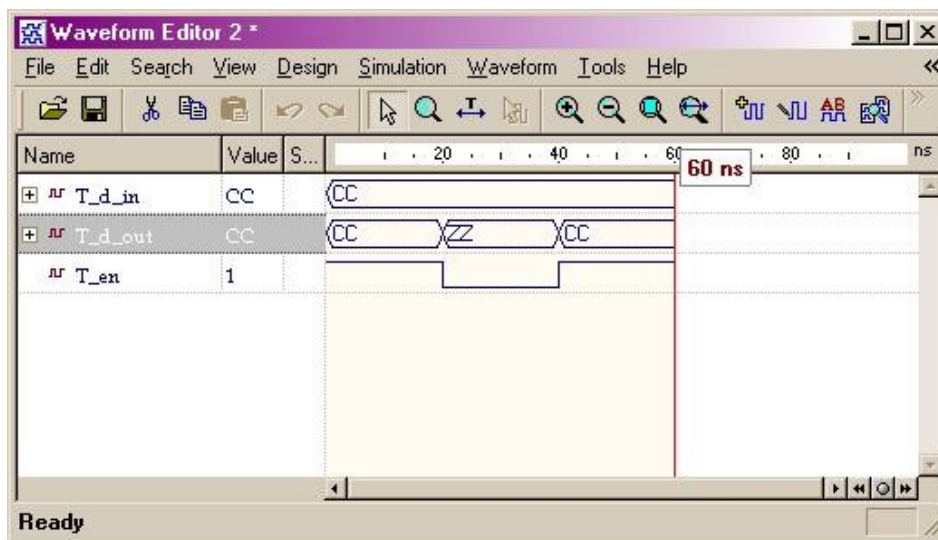
```
end TB;
```

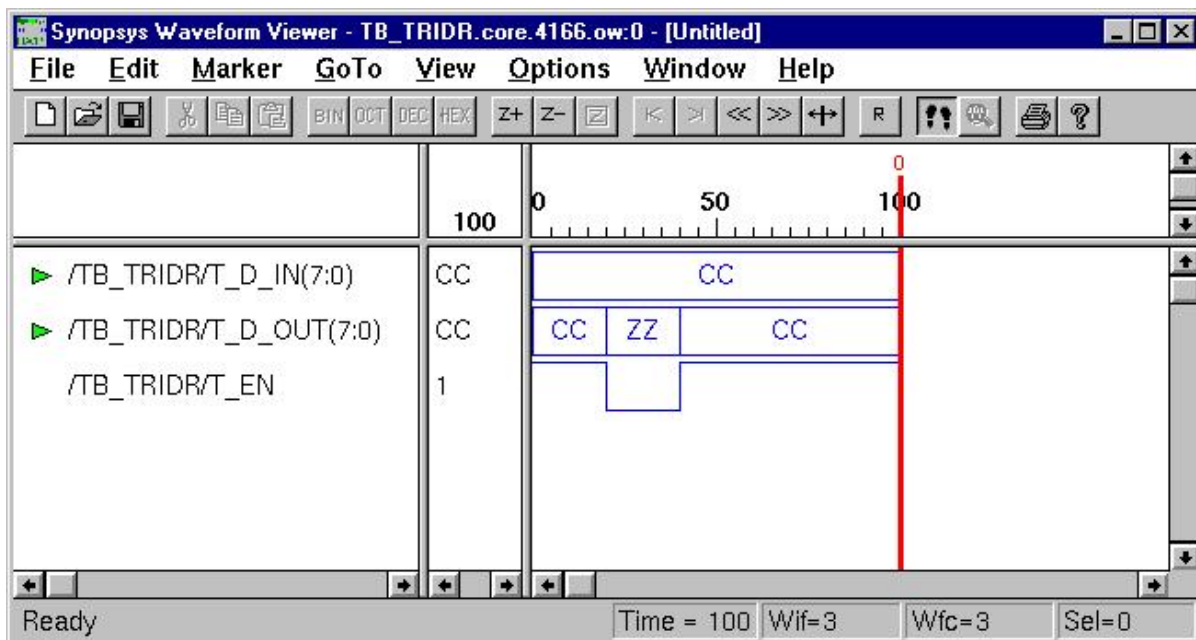
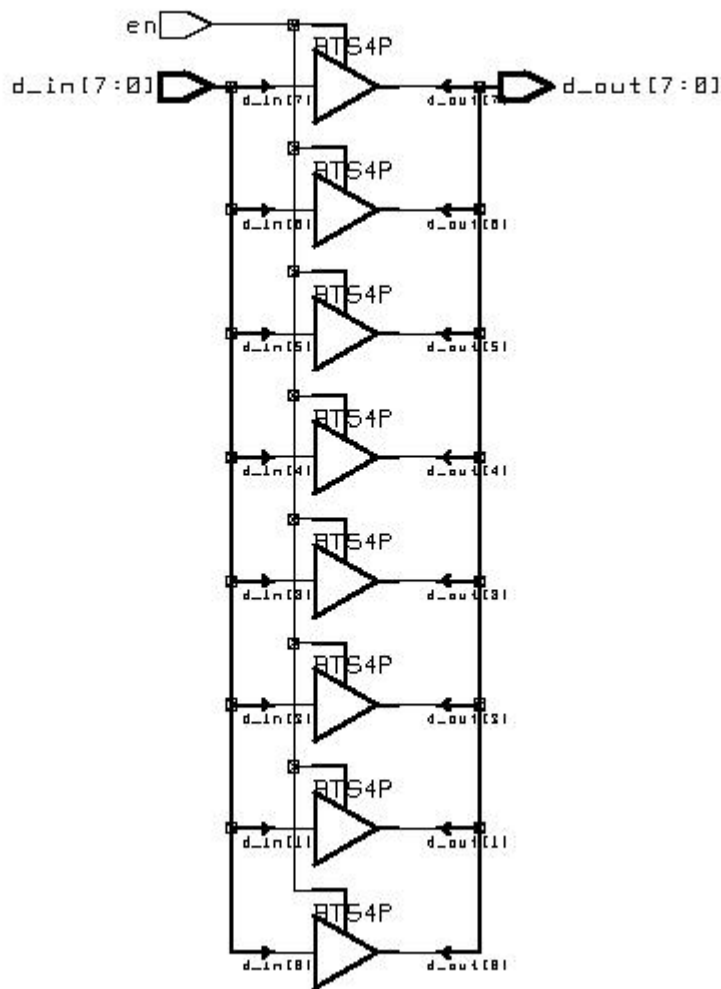
configuration CFG_TB of TB_tridr is

```
    for TB
```

```
        end for;
```

```
end CFG_TB;
```





3. Signal vs. Variable

Signals are used to connect the design components and must carry the information between statements of the design. On the other hand, variables are used within process to compute certain values. the following example shows their difference:

```
-----  
-- Signal vs. Variable (sig_var.vhd)  
-- Look at the outputs in simulation waveform  
-- for same computation, we get two different results  
--  
-- by Weijun Zhang, 05/2001  
-----
```

```
library ieee;  
use ieee.std_logic_1164.all;  
  
entity sig_var is  
port(   d1, d2, d3:    in std_logic;  
        res1, res2:    out std_logic);  
end sig_var;
```

```
architecture behv of sig_var is
```

```
    signal sig_s1: std_logic;
```

```
begin
```

```
    proc1: process(d1,d2,d3)
```

```
        variable var_s1: std_logic;
```

```
    begin
```

```
        var_s1 := d1 and d2;
```

```
        res1 <= var_s1 xor d3;
```

```
    end process;
```

```
    proc2: process(d1,d2,d3)
```

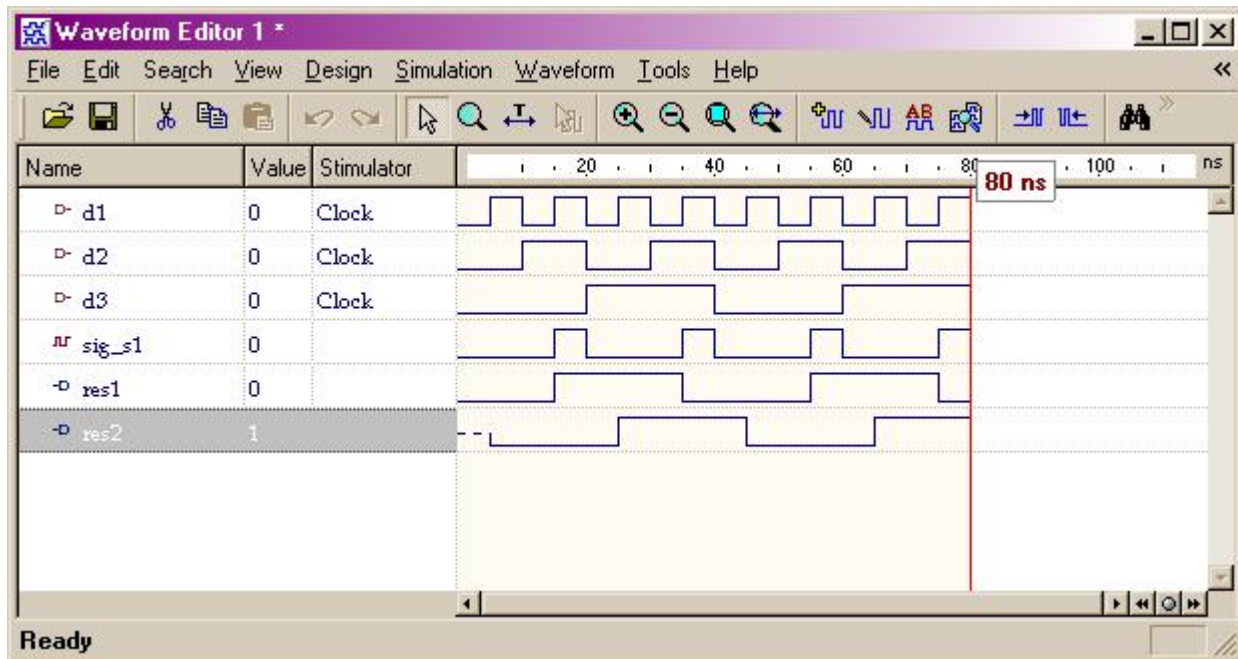
```
    begin
```

```
        sig_s1 <= d1 and d2;
```

```
        res2 <= sig_s1 xor d3;
```

```
    end process;
```

```
end behv;
```



PART B

Typical Combinational Components

The following behavior style codes demonstrate the concurrent and sequential capabilities of VHDL. The concurrent statements are written within the body of architecture. They include concurrent signal assignment, concurrent process and component instantiations (port map statement). Sequential statements are written within a process statement, function or procedure. Sequential statements include case statement, if-then-else statement and loop statement.

1. Multiplexor

```
-- VHDL code for 4:1 multiplexor
-- (ESD book figure 2.5)
-- by Weijun Zhang, 04/2001
--
-- Multiplexor is a device to select different
-- inputs to outputs. we use 3 bits vector to
-- describe its I/O ports
```

```
library ieee;
use ieee.std_logic_1164.all;
```

```
-----

entity Mux is
port(   I3:    in std_logic_vector(2 downto 0);
        I2:    in std_logic_vector(2 downto 0);
        I1:    in std_logic_vector(2 downto 0);
        I0:    in std_logic_vector(2 downto 0);
        S:     in std_logic_vector(1 downto 0);
        O:     out std_logic_vector(2 downto 0)
);
end Mux;
```

```
-----

architecture behv1 of Mux is
begin
    process(I3,I2,I1,I0,S)
    begin

        -- use case statement
        case S is
            when "00" => O <= I0;
            when "01" => O <= I1;
            when "10" => O <= I2;
            when "11" => O <= I3;
            when others =>      O <= "ZZZ";
        end case;

    end process;
end behv1;
```

```
architecture behv2 of Mux is
begin

    -- use when.. else statement
    O <= I0 when S="00" else
        I1 when S="01" else
        I2 when S="10" else
        I3 when S="11" else
        "ZZZ";

end behv2;
```

TEST BENCH VHDL CODE FOR MULTIPLEXOR

```
-----  
-- Test Bench for Multiplexer (ESD figure 2.5)  
-- by Weijun Zhang, 04/2001  
--  
-- four operations are tested in this example.  
-----
```

```
library IEEE;  
use IEEE.std_logic_1164.all;  
use IEEE.std_logic_arith.all;
```

```
entity Mux_TB is                                -- empty entity  
end Mux_TB;
```

```
-----  
architecture TB of Mux_TB is
```

```
    -- initialize the declared signals  
    signal T_I3: std_logic_vector(2 downto 0):="000";  
    signal T_I2: std_logic_vector(2 downto 0):="000";  
    signal T_I1: std_logic_vector(2 downto 0):="000";  
    signal T_I0: std_logic_vector(2 downto 0):="000";  
    signal T_O: std_logic_vector(2 downto 0);  
    signal T_S: std_logic_vector(1 downto 0);
```

```
    component Mux  
    port( I3:    in std_logic_vector(2 downto 0);  
          I2:    in std_logic_vector(2 downto 0);  
          I1:    in std_logic_vector(2 downto 0);  
          I0:    in std_logic_vector(2 downto 0);  
          S:     in std_logic_vector(1 downto 0);  
          O:     out std_logic_vector(2 downto 0)  
    );  
    end component;
```

```
begin
```

```
    U_Mux: Mux port map (T_I3, T_I2, T_I1, T_I0, T_S, T_O);
```

```
    process
```

```
        variable err_cnt: integer :=0;
```

```
    begin
```

```
        T_I3 <= "001";           -- I0-I3 are different signals  
        T_I2 <= "010";
```

```

T_I1 <= "101";
T_I0 <= "111";

-- case select equal "00"
wait for 10 ns;
T_S <= "00";
wait for 1 ns;
assert (T_O="111") report "Error Case 0" severity error;
if (T_O/="111") then
    err_cnt := err_cnt+1;
end if;

-- case select equal "01"
wait for 10 ns;
T_S <= "01";
wait for 1 ns;
assert (T_O="101") report "Error Case 1" severity error;
if (T_O/="101") then
    err_cnt := err_cnt+1;
end if;

-- case select equal "10"
wait for 10 ns;
T_S <= "10";
wait for 1 ns;
assert (T_O="010") report "Error Case 2" severity error;
if (T_O/="010") then
    err_cnt := err_cnt+1;
end if;

-- case select equal "11"
wait for 10 ns;
T_S <= "11";
wait for 1 ns;
assert (T_O="001") report "Error Case 3" severity error;
if (T_O/="001") then
    err_cnt := err_cnt+1;
end if;

-- case equal "11"
wait for 10 ns;
T_S <= "UU";

-- summary of all the tests
if (err_cnt=0) then
    assert (false)
    report "Testbench of Mux completed sucessfully!"
    severity note;
else
    assert (true)

```

```

        report "Something wrong, try again!"
        severity error;
    end if;

```

```

    wait;

```

```

end process;

```

```

end TB;

```

```

configuration CFG_TB of Mux_TB is

```

```

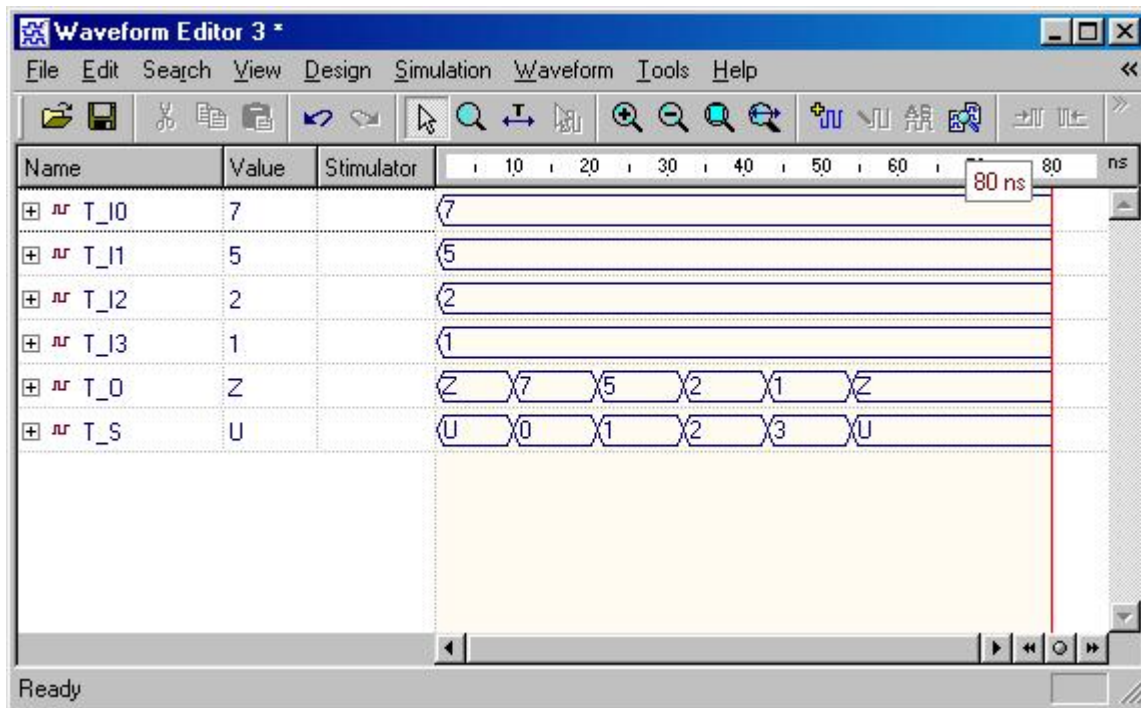
    for TB
    end for;

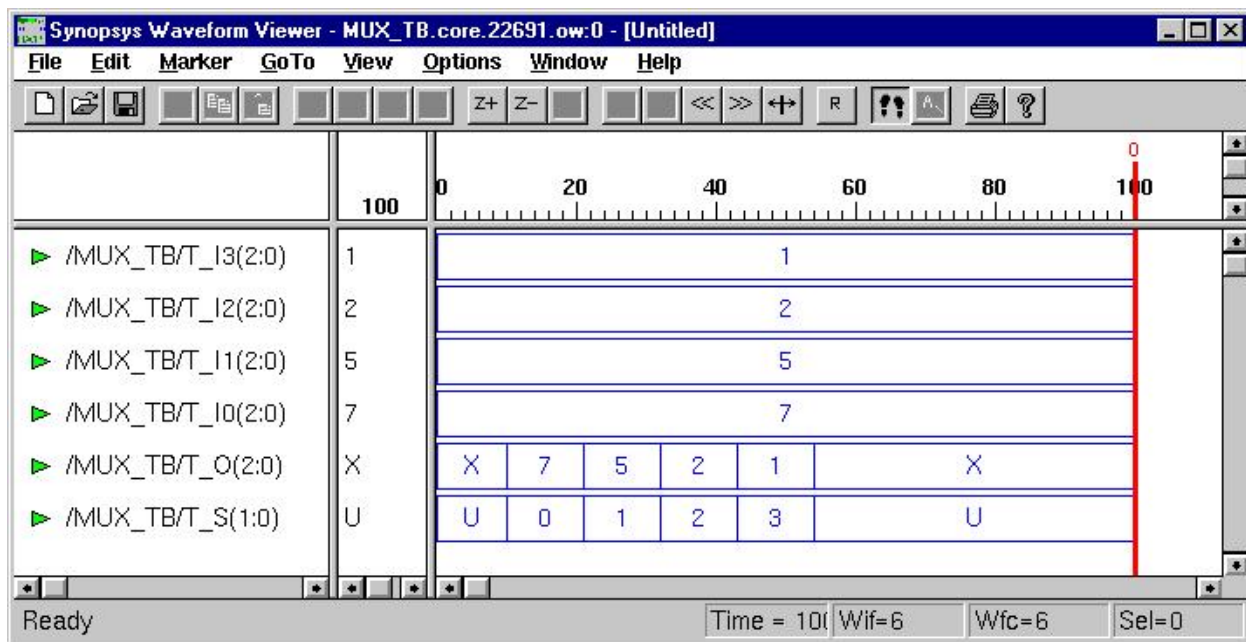
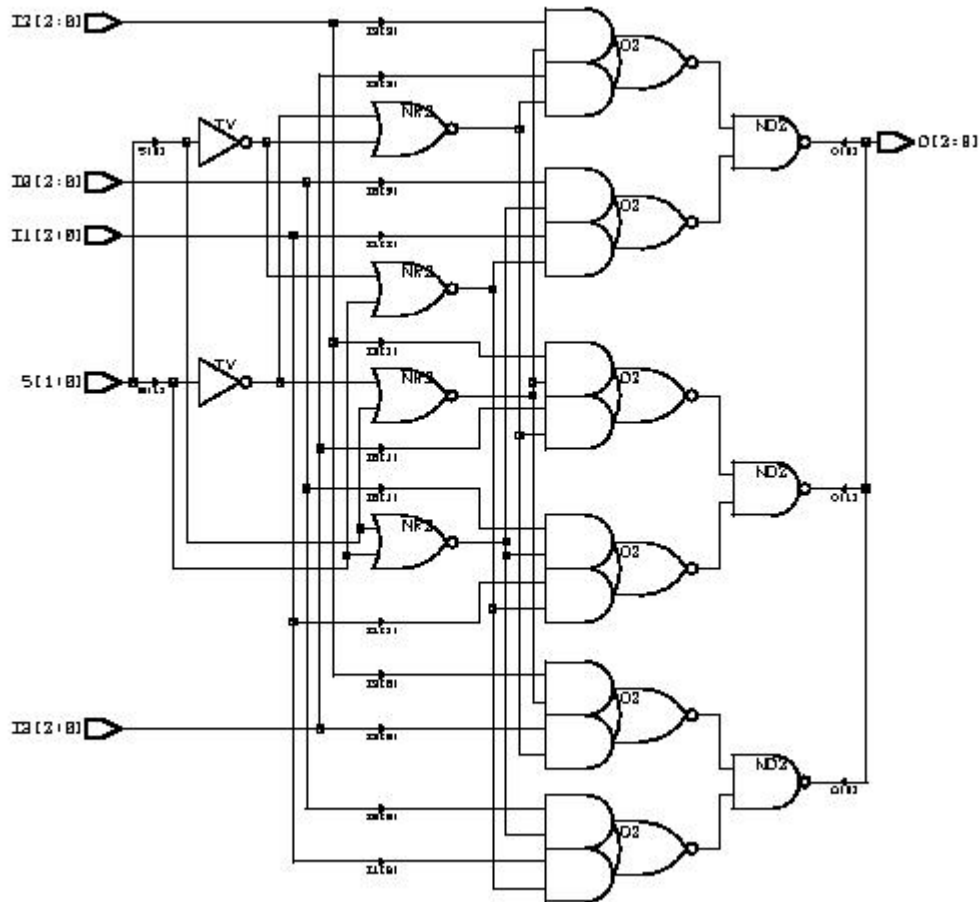
```

```

end CFG_TB;

```





2. DECODER VHDL CODE

```
-----  
-- 2:4 Decoder (ESD figure 2.5)  
-- by Weijun Zhang, 04/2001  
--  
-- decoder is a kind of inverse process  
-- of multiplexor  
-----
```

```
library ieee;  
use ieee.std_logic_1164.all;
```

```
-----  
entity DECODER is  
port(   I:      in std_logic_vector(1 downto 0);  
        O:      out std_logic_vector(3 downto 0)  
);  
end DECODER;
```

```
-----  
architecture behv of DECODER is  
begin
```

```
    -- process statement
```

```
    process (I)  
    begin
```

```
        -- use case statement
```

```
        case I is  
            when "00" => O <= "0001";  
            when "01" => O <= "0010";  
            when "10" => O <= "0100";  
            when "11" => O <= "1000";  
            when others => O <= "XXXX";  
        end case;
```

```
    end process;
```

```
end behv;
```

```
architecture when_else of DECODER is  
begin
```

```
    -- use when..else statement
```

```

O <= "0001" when I = "00" else
    "0010" when I = "01" else
    "0100" when I = "10" else
    "1000" when I = "11" else
    "XXXX";

end when_else;

```

TESTBENCH VHDL CODE FOR DECODER

```

-----
-- Test Bench for Decoder (ESD figure 2.5)
-- by Weijun Zhang, 04/2001
--
-- four cases are tested here.
-----

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;

entity DECODER_TB is                                -- entity declaration
end DECODER_TB;

architecture TB of DECODER_TB is

    signal T_I: std_logic_vector(1 downto 0):="00";
    signal T_O: std_logic_vector(3 downto 0);

    -- declare the component
    component DECODER
    port( I:      in std_logic_vector(1 downto 0);
          O:      out std_logic_vector(3 downto 0)
    );
    end component;

begin

    U_DECODER: DECODER port map (T_I, T_O);

    process

        -- variable should be declared within process
        variable err_cnt : integer := 0;

    begin

```

```

-- case "00"
wait for 10 ns;
T_I <= "00";
wait for 1 ns;
assert (T_O="0001") report "Error Case 0"
severity error;
if (T_O/="0001") then
    err_cnt := err_cnt + 1;
end if;

-- case "01"
wait for 10 ns;
T_I <= "01";
wait for 1 ns;
assert (T_O="0010") report "Error Case 1"
severity error;
if (T_O/="0010") then
    err_cnt := err_cnt + 1;
end if;

-- case "10"
wait for 10 ns;
T_I <= "10";
wait for 1 ns;
assert (T_O="0100") report "Error Case 2"
severity error;
if (T_O/="0100") then
    err_cnt := err_cnt + 1;
end if;

-- case "11"
wait for 10 ns;
T_I <= "11";
wait for 1 ns;
assert (T_O="1000") report "Error Case 3"
severity error;
if (T_O/="1000") then
    err_cnt := err_cnt + 1;
end if;

-- case "11"
wait for 10 ns;
T_I <= "UU";

-- summary of all the tests
if (err_cnt=0) then
    assert false
    report "Testbench of Adder completed successfully!"
    severity note;
else

```

```

    assert true
    report "Something wrong, try again"
    severity error;
end if;

```

```

wait;

```

```

end process;

```

```

end TB;

```

```

configuration CFG_TB of DECODER_TB is

```

```

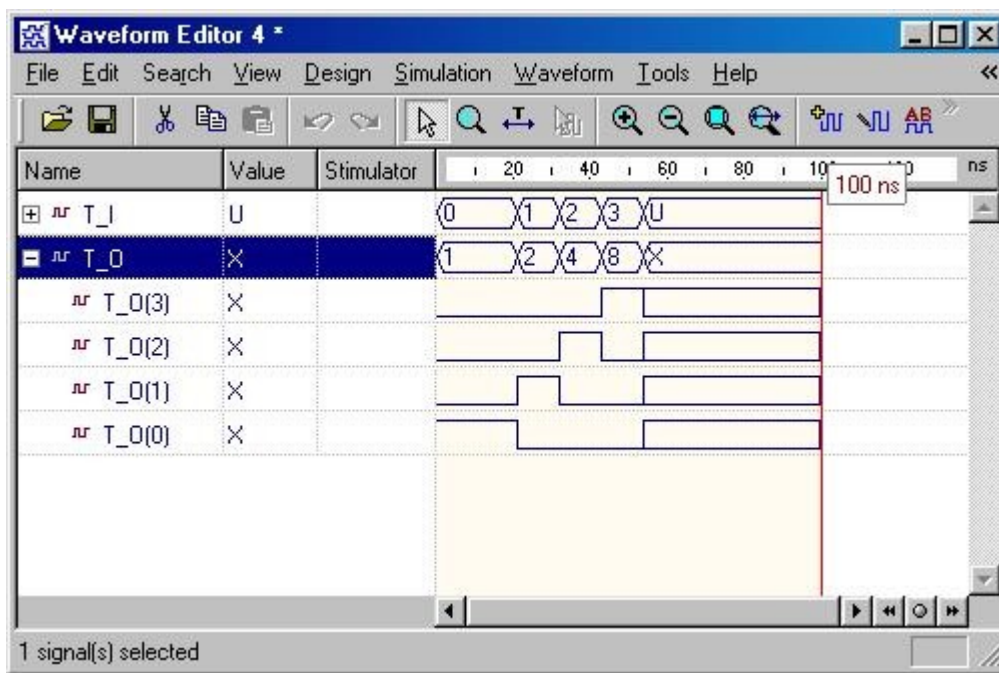
    for TB
    end for;

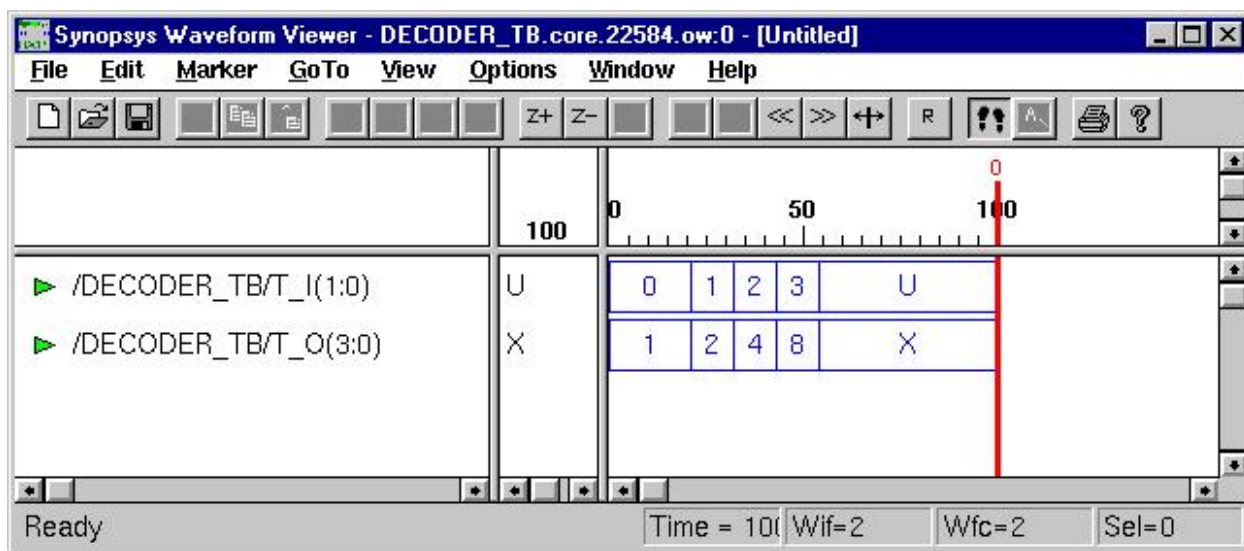
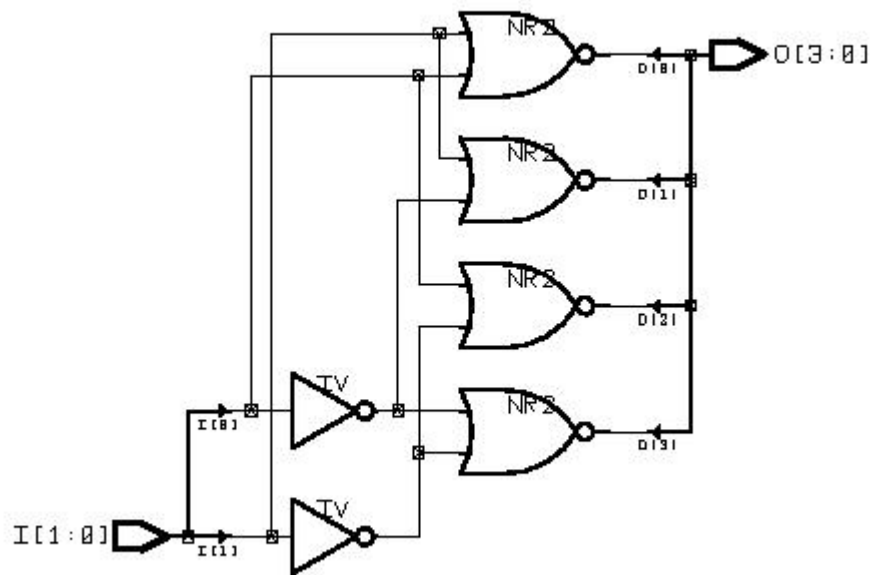
```

```

end CFG_TB;

```





3. ADDER VHDL CODE

```

-- VHDL code for n-bit adder (ESD figure 2.5)
-- by Weujun Zhang, 04/2001
--
-- function of adder:
-- A plus B to get n-bit sum and 1 bit carry
-- we may use generic statement to set the parameter
-- n of the adder.

```

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
```

```
-----

entity ADDER is
```

```
generic(n: natural :=2);
port(   A:      in std_logic_vector(n-1 downto 0);
        B:      in std_logic_vector(n-1 downto 0);
        carry:  out std_logic;
        sum:    out std_logic_vector(n-1 downto 0)
);
```

```
end ADDER;
```

```
-----

architecture behv of ADDER is
```

```
-- define a temporary signal to store the result
```

```
signal result: std_logic_vector(n downto 0);
```

```
begin
```

```
-- the 3rd bit should be carry
```

```
result <= ('0' & A)+('0' & B);
sum <= result(n-1 downto 0);
carry <= result(n);
```

```
end behv;
```

```
-----



## TEST BENCH VHDL CODE FOR ADDER


```

```
-----

-- Test Bench for 2-bit Adder (ESD figure 2.5)
```

```
-- by Weijun Zhang, 04/2001
```

```
--
```

```
-- five cases are tested in this example
```

```
-----

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
```

```
use ieee.std_logic_arith.all;
```

```
entity ADDER_TB is                                -- entity declaration
end ADDER_TB;
```

```
architecture TB of ADDER_TB is
```

```
    component ADDER is
    port( A:      in std_logic_vector(1 downto 0);
          B:      in std_logic_vector(1 downto 0);
          carry:  out std_logic;
          sum:    out std_logic_vector(1 downto 0)
    );
    end component;
```

```
    signal A, B:  std_logic_vector(1 downto 0);
    signal carry: std_logic;
    signal sum:   std_logic_vector(1 downto 0);
```

```
begin
```

```
    U_ADDER: ADDER port map (A, B, carry, sum);
```

```
process
```

```
    variable err_cnt: integer :=0;
```

```
begin
```

```
    -- case 1
    A <= "00";
    B <= "00";
    wait for 10 ns;
    assert (sum="00") report "Sum Error!" severity error;
    assert (carry='0') report "Carry Error!" severity error;
    if (sum/="00" or carry/='0') then
        err_cnt:=err_cnt+1;
    end if;
```

```
    -- case 2
    A <= "11";
    B <= "11";
    wait for 10 ns;
```

```
    assert (sum="10") report "Sum Error!" severity error;
    assert (carry='1') report "Carry Error!" severity error;
    if (sum/="10" or carry/='1') then
        err_cnt:=err_cnt+1;
    end if;
```

```

-- case 3
A <= "01";
B <= "10";
wait for 10 ns;

assert (sum="11") report "Sum Error!" severity error;
assert (carry='0') report "Carry Error!" severity error;
if (sum/="11" or carry/='0') then
    err_cnt:=err_cnt+1;
end if;

-- case 4
A <= "10";
B <= "01";
wait for 10 ns;

assert (sum="11") report "Sum Error!" severity error;
assert (carry='0') report "Carry Error!" severity error;
if (sum/="11" or carry/='0') then
    err_cnt:=err_cnt+1;
end if;

-- case 5
A <= "01";
B <= "01";
wait for 10 ns;
assert (sum="10") report "Sum Error!" severity error;
assert (carry='0') report "Carry Error!" severity error;
if (sum/="10" or carry/='0') then
    err_cnt:=err_cnt+1;
end if;

-- summary of testbench
if (err_cnt=0) then
    assert false
    report "Testbench of Adder completed successfully!"
    severity note;
else
    assert true
    report "Something wrong, try again"
    severity error;
end if;

wait;

end process;

```

```

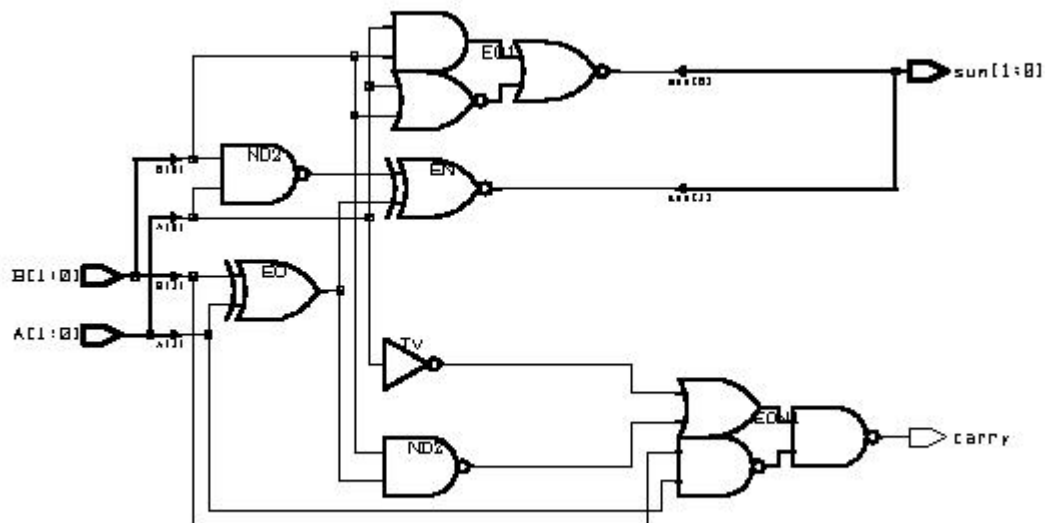
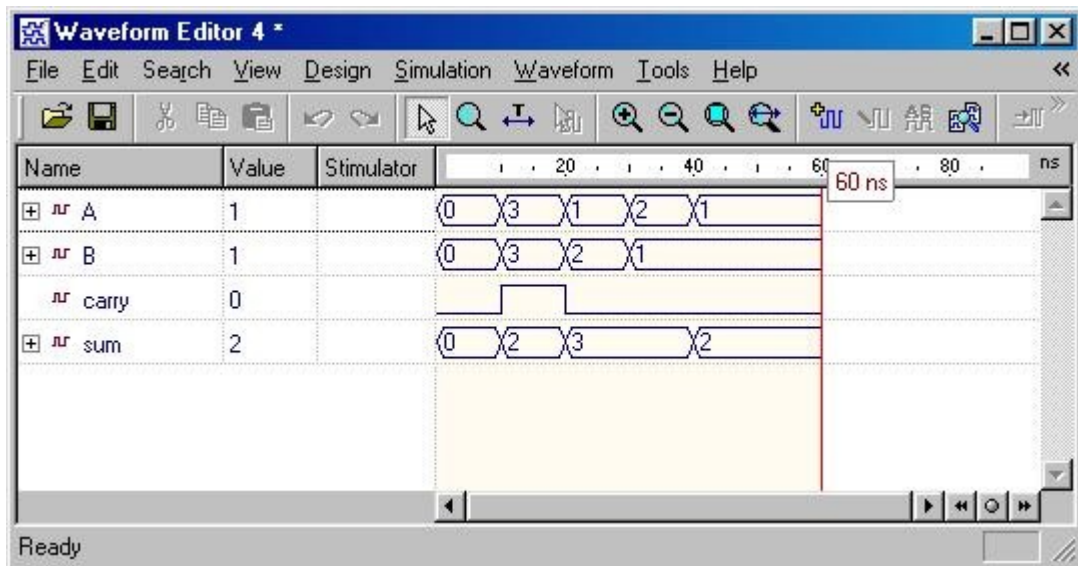
end TB;

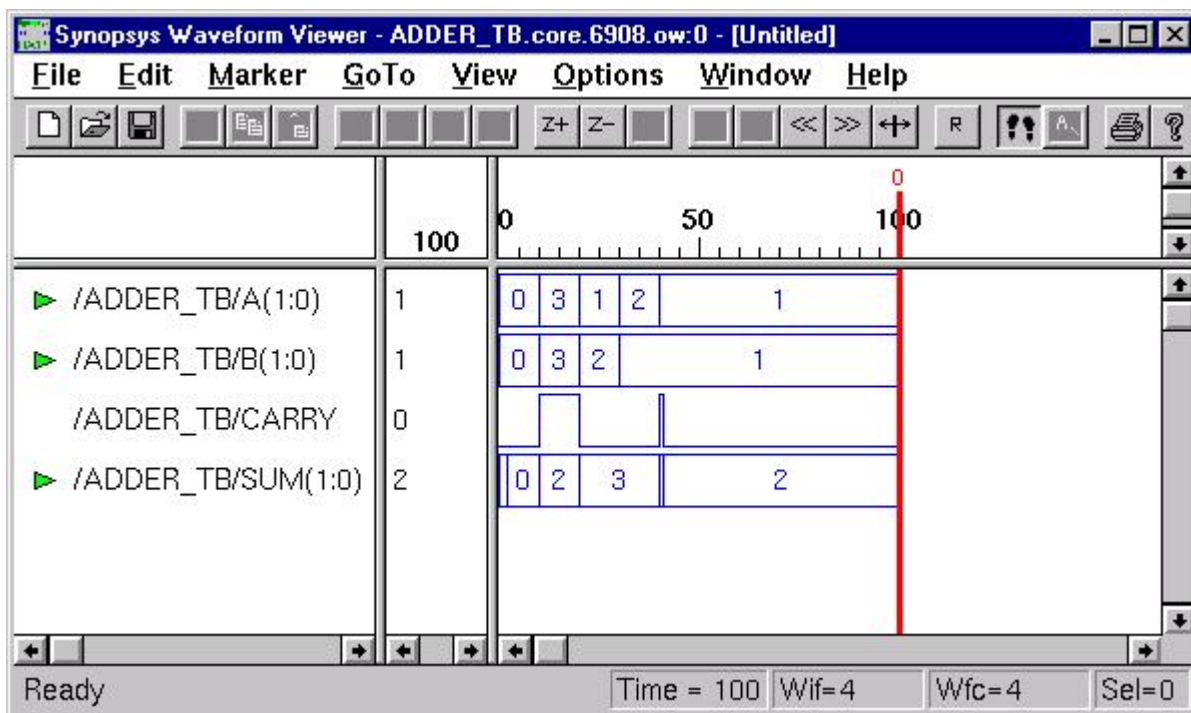
```

```

configuration CFG_TB of ADDER_TB is
    for TB
        end for;
end CFG_TB;

```





4. COMPARATOR VHDL CODE

```

-----
-- n-bit Comparator (ESD book figure 2.5)
-- by Weijun Zhang, 04/2001
--
-- this simple comparator has two n-bit inputs &
-- three 1-bit outputs
-----

```

```

library ieee;
use ieee.std_logic_1164.all;
-----

```

entity Comparator is

```

generic(n: natural :=2);
port(  A:      in std_logic_vector(n-1 downto 0);
       B:      in std_logic_vector(n-1 downto 0);
       less:    out std_logic;
       equal:   out std_logic;
       greater: out std_logic
);
end Comparator;
-----

```

architecture behv of Comparator is

begin

```
process(A,B)
begin
  if (A<B) then
    less <= '1';
    equal <= '0';
    greater <= '0';
  elsif (A=B) then
    less <= '0';
    equal <= '1';
    greater <= '0';
  else
    less <= '0';
    equal <= '0';
    greater <= '1';
  end if;
end process;
```

end behv;

TEST BENCH VHDL CODE FOR COMPARATOR

```
-- Test Bench for 2-bit Comparator (ESD figure 2.5)
-- by Weijun Zhang, 04/2001
--
-- nine cases are tested here
```

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.std_logic_arith.all;
```

```
entity Comparator_TB is                                -- entity declaration
end Comparator_TB;
```

architecture TB of Comparator_TB is

component Comparator

```

port( A:          in std_logic_vector(1 downto 0);
      B:          in std_logic_vector(1 downto 0);
      less:       out std_logic;
      equal:      out std_logic;
      greater:out std_logic
);
end component;

```

```

signal A, B: std_logic_vector(1 downto 0):="00";
signal less, equal, greater: std_logic;

```

```
begin
```

```
Unit: Comparator port map (A, B, less, equal, greater);
```

```
process
```

```
variable err_cnt: integer :=0;
```

```
begin
```

```

-- Case 1 (using the loop statement)
A <= "11";
B <= "00";
for i in 0 to 2 loop
    wait for 10 ns;
    assert (greater='1') report "Comparison Error detected!"
    severity error;
    if (greater/='1') then
        err_cnt:=err_cnt+1;
    end if;
    B <= B + '1';
end loop;

```

```

-- Case 2 (using the loop statement)
A <= "00";
B <= "01";
for i in 0 to 2 loop
    wait for 10 ns;
    assert (less='1') report "Comparison Error detected!"
    severity error;
    if (less/='1') then
        err_cnt:=err_cnt+1;
    end if;
    B <= B + '1';
end loop;

```

```

-- Case 3
A <= "01";
B <= "01";

```

```

wait for 10 ns;
assert (equal='1') report "Comparison Error detected!"
severity error;
if (equal/= '1') then
    err_cnt:=err_cnt+1;
end if;

```

```

-- summary of all the tests
if (err_cnt=0) then
    assert false
    report "Testbench of Adder completed successfully!"
    severity note;
else
    assert true
    report "Something wrong, try again"
    severity error;
end if;

```

```
wait;
```

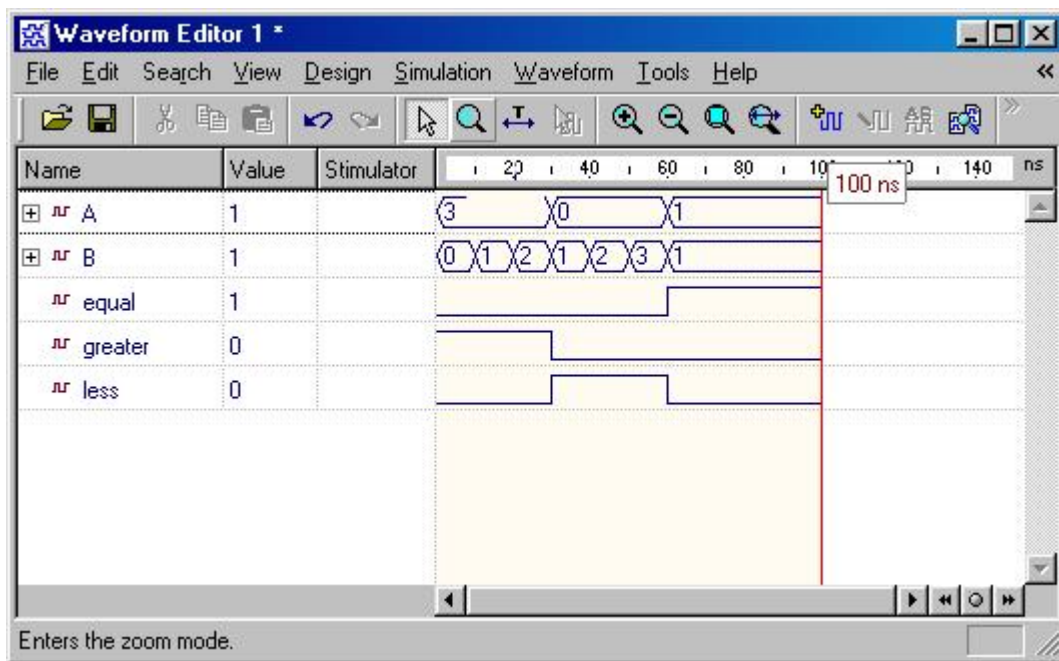
```
end process;
```

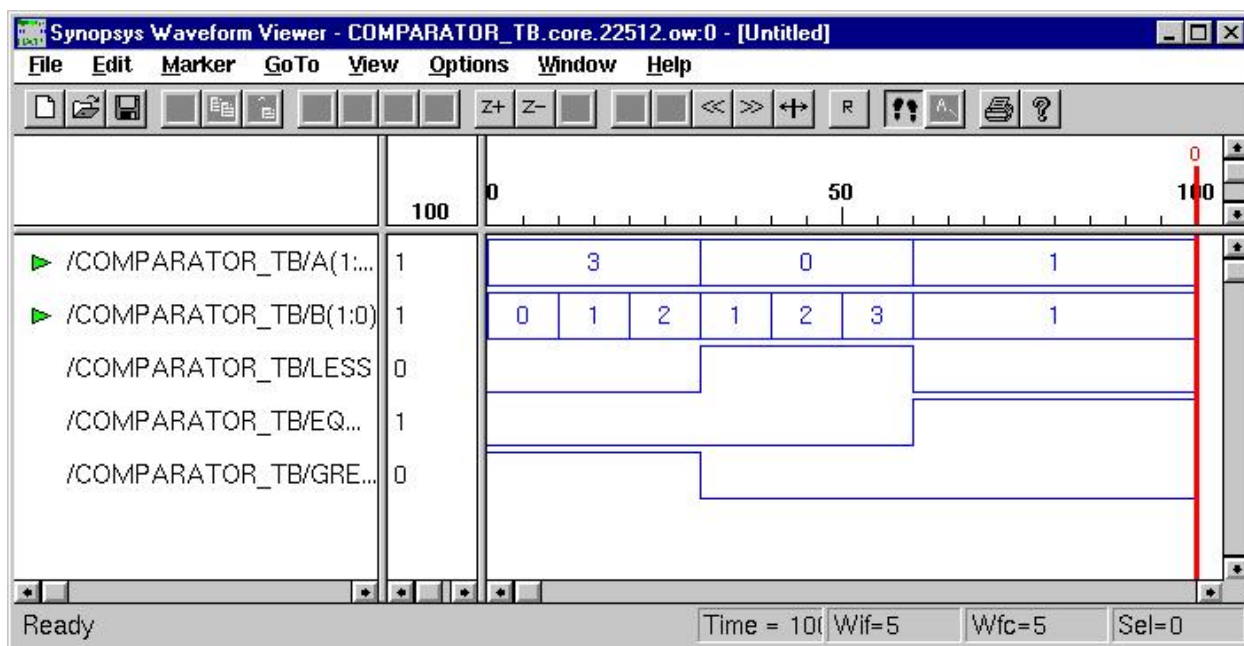
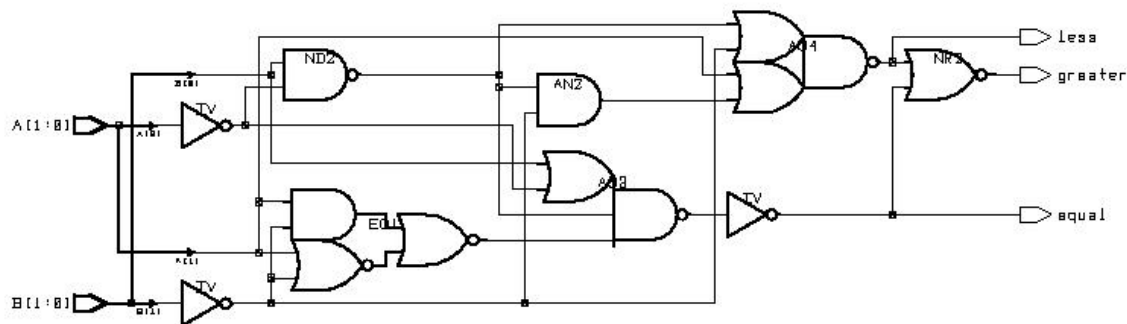
```
end TB;
```

```
configuration CFG_TB of Comparator_TB is
```

```
    for TB
    end for;
```

```
end CFG_TB;
```





5. ARITHMETIC LOGIC UNIT (ALU) VHDL CODE

- Simple ALU Module (ESD book Figure 2.5)
- by Weijun Zhang, 04/2001
-
- ALU stands for arithmetic logic unit.
- It perform multiple operations according to
- the control bits.
- we use 2's complement subtraction in this example
- two 2-bit inputs & carry-bit ignored

library ieee;

```
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.std_logic_arith.all;
```

entity ALU is

```
port(  A:    in std_logic_vector(1 downto 0);
       B:    in std_logic_vector(1 downto 0);
       Sel:  in std_logic_vector(1 downto 0);
       Res:  out std_logic_vector(1 downto 0)
);
```

end ALU;

architecture behv of ALU is
begin

```
    process(A,B,Sel)
    begin
```

```
        -- use case statement to achieve
        -- different operations of ALU
```

```
        case Sel is
            when "00" =>
                Res <= A + B;
            when "01" =>
                Res <= A + (not B) + 1;
            when "10" =>
                Res <= A and B;
            when "11" =>
                Res <= A or B;
            when others =>
                Res <= "XX";
        end case;
```

```
    end process;
```

```
end behv;
```

TEST BENCH VHDL CODE FOR ALU

```
-----  
-- Test Bench for ALU design (ESD figure 2.5)  
-- by Weijun Zhang, 04/2001  
--  
-- we illustrate how to use package and procedure in this example  
-- it seems a kind of complex testbench for this simple module,  
-- the method, however, makes huge circuit testing more complete,  
-- convenient and manageable  
-----
```

```
library ieee;  
use ieee.std_logic_1164.all;  
use ieee.std_logic_unsigned.all;  
use ieee.std_logic_arith.all;  
  
-- define constant, signal and procedure within package for ALU  
  
package ALU_package is  
  
    constant INTERVAL: TIME := 8 ns;  
  
    signal sig_A, sig_B: std_logic_vector(1 downto 0);  
    signal sig_Sel: std_logic_vector(1 downto 0);  
    signal sig_Res: std_logic_vector(1 downto 0);  
  
    procedure load_data(signal A, B: out std_logic_vector(1 downto 0);  
        signal Sel: out std_logic_vector(1 downto 0));  
  
    procedure check_data(signal Sel: out std_logic_vector( 1 downto 0));  
  
end ALU_package;  
  
-- put all the procedure descriptions within package body  
  
package body ALU_package is  
  
    procedure load_data (signal A, B: out std_logic_vector(1 downto 0);  
        signal Sel: out std_logic_vector(1 downto 0) ) is  
    begin  
        A <= sig_A;  
        B <= sig_B;  
        Sel <= sig_Sel;  
    end load_data;  
  
    procedure check_data (signal Sel: out std_logic_vector( 1 downto 0)) is  
    begin  
        Sel <= sig_Sel;
```



```

        if (sig_Sel="00") then
            assert(sig_Res = (sig_A + sig_B))
            report "Error detected in Addition!"
            severity warning;
        elsif (sig_Sel="01") then
            assert(sig_Res = (sig_A - sig_B))
            report "Error detected in Subtraction!"
            severity warning;
        elsif (sig_Sel="10") then
            assert(sig_Res = (sig_A and sig_B))
            report "AND Operation Error!"
            severity warning;
        elsif (sig_Sel="11") then
            assert(sig_Res = (sig_A or sig_B))
            report "OR operation Error!"
            severity warning;
        end if;
    end check_data;

end ALU_package;

-- Test Bench code for ALU
-----

library IEEE;
use IEEE.std_logic_1164.all;
use work.ALU_package.all;

entity ALU_TB is                                -- entity declaration
end ALU_TB;

architecture TB of ALU_TB is

    component ALU
    port( A:      in std_logic_vector(1 downto 0);
          B:      in std_logic_vector(1 downto 0);
          Sel:    in std_logic_vector(1 downto 0);
          Res:    out std_logic_vector(1 downto 0)
    );
    end component;

    signal A, B, Res: std_logic_vector(1 downto 0):="00";
    signal Sel: std_logic_vector(1 downto 0);

begin

    U_ALU: ALU port map (A, B, Sel, Res);

    process
    begin

```

```
sig_A <= "10";  
sig_B <= "01";
```

```
sig_Sel <= "00";           -- case 1: Addition  
wait for 1 ns;  
load_data(A, B, Sel);  
wait for 1 ns;  
sig_Res <= Res;  
wait for INTERVAL;  
check_data(Sel);
```

```
sig_Sel <= "01";           -- case 2: subtraction  
wait for 1 ns;  
load_data(A, B, Sel);  
wait for 1 ns;  
sig_Res <= Res;  
wait for INTERVAL;  
check_data(Sel);
```

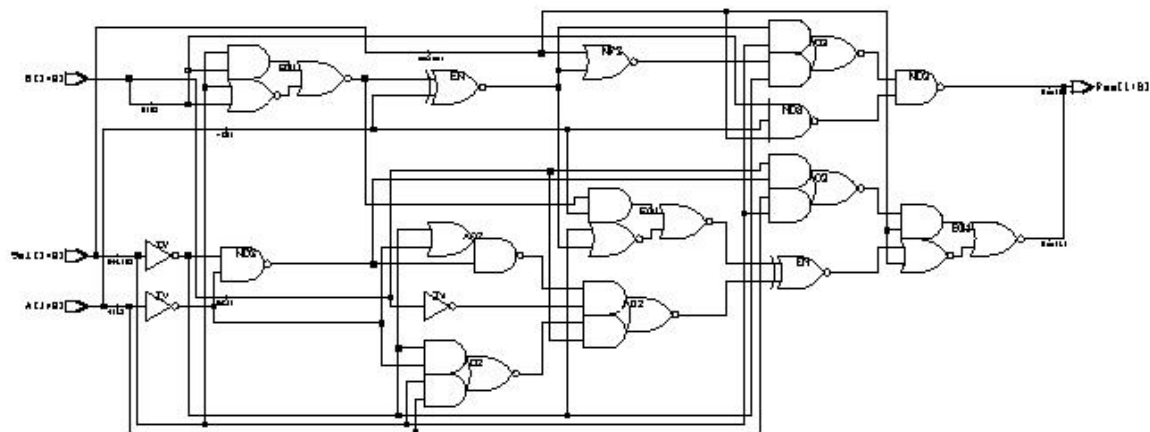
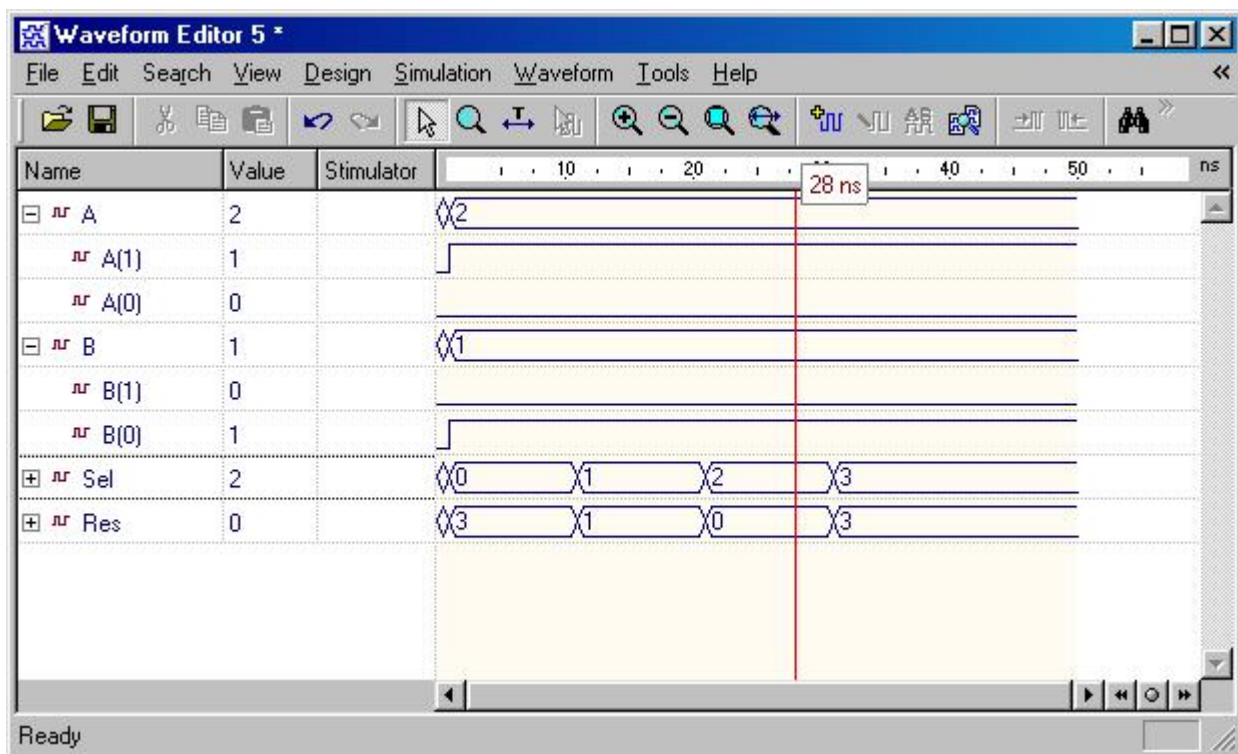
```
sig_Sel <= "10";           -- case 3: AND operation  
wait for 1 ns;  
load_data(A, B, Sel);  
wait for 1 ns;  
sig_Res <= Res;  
wait for INTERVAL;  
check_data(Sel);
```

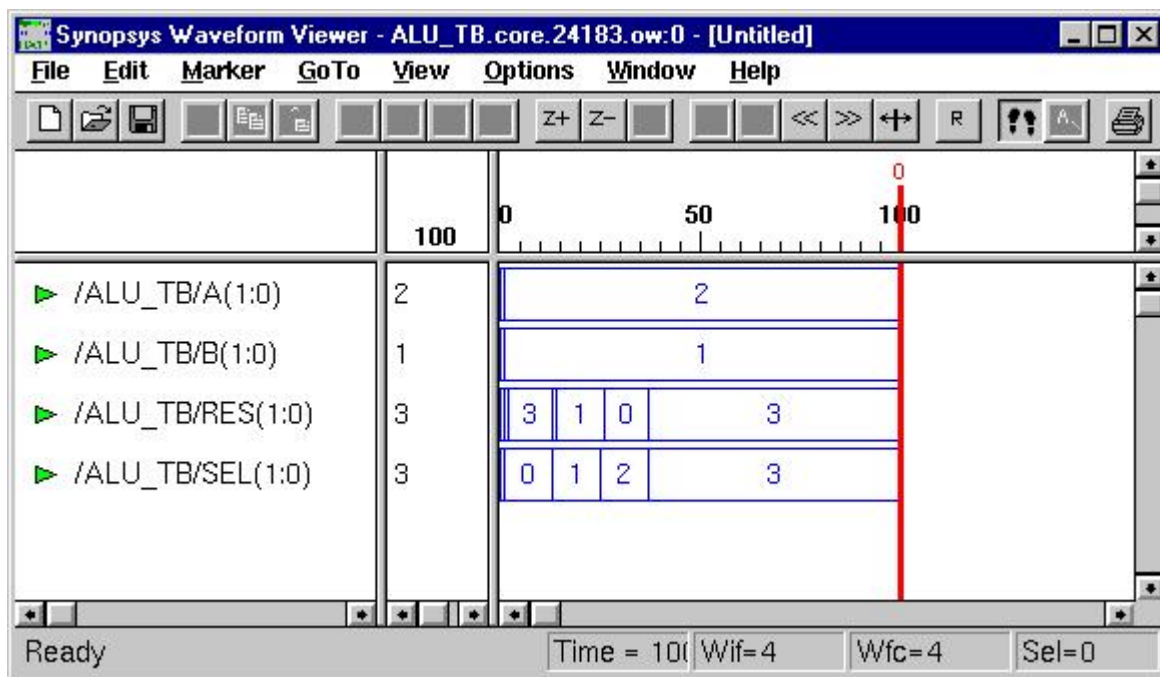
```
sig_Sel <= "11";           -- case 4: OR operation  
wait for 1 ns;  
load_data(A, B, Sel);  
wait for 1 ns;  
sig_Res <= Res;  
wait for INTERVAL;  
check_data(Sel);  
wait;
```

```
end process;
```

```
end TB;
```

```
-----  
configuration CFG_TB of ALU_TB is  
    for TB  
        end for;  
end CFG_TB;  
-----
```





6. MULTIPLIER VHDL CODE

```

-----
-- Example of doing multiplication showing
-- (1) how to use variable with in process
-- (2) how to use for loop statement
-- (3) algorithm of multiplication
--
-- by Weijun Zhang, 05/2001
-----

```

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

```

```

-- two 4-bit inputs and one 8-bit outputs

```

```

entity multiplier is

```

```

port(  num1,num2:  in std_logic_vector(1 downto 0);
      product:      out std_logic_vector(3 downto 0)
);
end multiplier;

```

```

architecture behv of multiplier is

```

```

begin
process(num1, num2)

```

```

variable num1_reg: std_logic_vector(2 downto 0);
variable product_reg: std_logic_vector(5 downto 0);

begin

num1_reg := '0' & num1;
product_reg := "0000" & num2;

-- use variables doing computation
-- algorithm is to repeat shifting/adding
for i in 1 to 3 loop
    if product_reg(0)='1' then
        product_reg(5 downto 3) := product_reg(5 downto 3)
        + num1_reg(2 downto 0);
    end if;
    product_reg(5 downto 0) := '0' & product_reg(5 downto 1);
end loop;

-- assign the result of computation back to output signal
product <= product_reg(3 downto 0);

end process;

end behv;

```

TEST BENCH VHDL CODE FOR MULTIPLIER

```

-----
-- Test Bench for Mutiplier
-- by Weijun Zhang, 05/2001
-----

```

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity TB_multiplier is
end TB_multiplier;

architecture TB of TB_multiplier is

    component multiplier is
        port( num1, num2: in std_logic_vector(1 downto 0);
              product: out std_logic_vector(3 downto 0)
            );
    end component;

```

```
signal T_num1, T_num2: std_logic_vector(1 downto 0);
signal T_product: std_logic_vector(3 downto 0);

begin

    U_UT: multiplier port map (T_num1, T_num2, T_product);
```

```
process
begin

    T_num1 <= "11";
    T_num2 <= "11";
    wait for 20 ns;
    assert(T_product="1001") report "Error detected!"
    severity warning;

    T_num1 <= "01";
    T_num2 <= "00";
    wait for 20 ns;
    assert(T_product="0000") report "Error detected!"
    severity warning;

    T_num1 <= "10";
    T_num2 <= "10";
    wait for 20 ns;
    assert(T_product="0100") report "Error detected!"
    severity warning;

    T_num1 <= "11";
    T_num2 <= "10";
    wait for 20 ns;
    assert(T_product="0110") report "Error detected!"
    severity warning;

    wait;
```

```
end process;
```

```
end TB;
```

```
-----
configuration CFG_TB of TB_multiplier is
    for TB
        end for;
end CFG_TB;
-----
```

